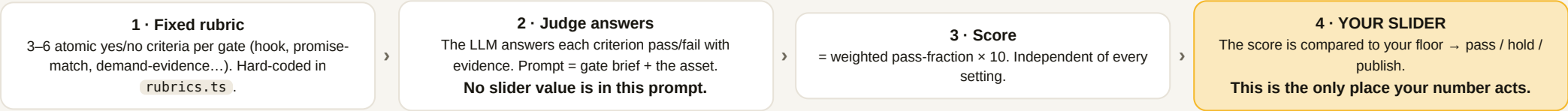


Quality Gate Scores — what the sliders actually do

Settings → Quality gates. This explains, per knob, how adjusting the 0–10 value changes the autonomous agents — audited against the code (qc.ts, rubrics.ts, watch-gate.ts, competitive-judge.ts, guardrails.ts, engine.ts).

The headline (audited & confirmed): adjusting a gate's 0–10 score **does NOT change how the agents grade**. The agents grade against **fixed, hand-authored rubrics**; the 0–10 score is a weighted pass-fraction computed independently of your settings. Your sliders are **pass/hold lines** — they decide what a given score is *allowed to do* (pass, hold for you, auto-publish), not how strictly the judge assesses the work. Raise them → more videos hold for you; lower them → more flow through automatically.

How a grade is actually produced (and where your slider enters)



So to make the channel "stricter," a higher floor makes the *same* grades hold more often. It does not make the judge grade harder. (The one indirect exception — model escalation — is noted below.)

Every knob in Settings → Quality gates

SETTING (DEFAULT • RANGE)	TYPE	WHAT THE AGENT DOES	WHAT RAISING THE NUMBER DOES
ideaFloor 6 • 0–10	PASS LINE	The idea judge scores each scouted idea (audience-fit, demand-evidence, differentiation...).	More ideas fall below the line → auto-dismissed; fewer reach the Idea gate. Doesn't change how ideas are scored.
revisionWarnAt 3 • 1–10	BEHAVIOR	Counts your "Changes" (revision) rounds on a video.	Delays the "you're revising a lot" warning to a later round. (Newly exposed — was silently reset before; now round-trips.)
revisionHardCap 4 • 1–10	BEHAVIOR	Same counter.	Allows more revision rounds before the video is hard-held (protects spend).
qcLessonBelow 7 • 0–10	PASS LINE	Mines past low SCRIPT QC reviews into "lessons" fed to the writer.	Raising it treats more past reviews as lessons (more feedback into the writer); lowering it keeps only the worst.
coldStartMin 5 • 0–50	BEHAVIOR	Counts published-with-stats videos before performance ranking activates.	Waits for more real data before the optimizer ranks ideas by past performance. (Newly exposed — was silently reset before.)
seedVisionFloor 6 • 0–10	PASS LINE	Vision-scores a seed still <i>before</i> paying to animate it into video.	More stills get re-rolled before the expensive video step → higher visual bar, slightly more image spend, less wasted video spend.
varietyMinDistance 6 • 0–32	BEHAVIOR	Measures pHash distance between beat visuals.	Demands more visual difference between beats before accepting them (fewer look-alike shots). A pixel metric, not a graded score.

autofixThreshold 7 · 0–10	PASS LINE +MODEL	The auto-fix loop critiques the final render; frames under this get a fix pass.	More renders get an auto-fix attempt (higher quality bar, more re-render spend). Also picks the fixer-model tier near the value.
runFloor 7 · 0–10	PASS LINE	FINAL QC score on build-run / autopilot videos.	More finished videos hold for your review instead of auto-publishing.
runPublic 8 · 0–10	PASS LINE	FINAL QC score → posting privacy.	Requires a higher score to auto-publish Public (below it → Unlisted). Doesn't change the score.
factRiskMax 7 · 0–10	PASS LINE	The fact-check agent rates a script's claim-risk 0–10 (search-grounded).	Lowering it holds more scripts for fact concerns; raising it lets riskier claims through. The risk score itself is computed independently.
beatRelevanceFloor 6 · 0–10	PASS LINE	Vision-scores whether each beat's visual depicts its narration (also feeds Self-Watch #2).	More off-topic beat visuals get re-rolled / re-searched to match the script.
timingFloor 7 · 0–10	PASS LINE	Self-Watch timing dimension (coverage, pacing, dead-air, length) — structural, no LLM.	The timing dimension must clear a higher bar to pass the pre-publish watch gate.
watchBlockPublishBelow 5 · 0–10	PASS LINE	Overall Self-Watch score across all four dimensions.	Raising it holds more videos for a human even on Autopilot (the pre-publish hard floor).
competitiveFloor 6 · 0–10	PASS LINE	Self-Watch competitive-fit judge (hook vs niche winners, differentiation, packaging).	Requires a stronger competitive fit to pass; the judge's score is a binary-criteria pass-fraction, unchanged by the number.
transitionFloor 6 · 0–10	PASS LINE	Self-Watch transitions dimension (cut quality, open/close framing).	Transitions must clear a higher bar to pass.
watchTemporalEscalateAt 2 · 1–20	BEHAVIOR \$ SPEND	Counts suspect shot boundaries at Final.	A lower value escalates to the paid Tier-2 temporal (Gemini/TwelveLabs) video pass more often; a higher value rarely does. Spend lever.
playbookShadowMinEvidence 5 · 1–50	BEHAVIOR	Counts how often a proposed "shadow" quality lesson recurs.	Requires more evidence before a self-proposed criterion is allowed to actually gate videos (anti-reward-hacking).
playbookAutoGraduateConfidence 0.8 · 0–1	BEHAVIOR	Confidence attached to a shadow lesson.	Raises the confidence a self-learned criterion needs before it starts gating.
PASS LINE moves where a score passes/holds · BEHAVIOR controls a non-score behavior (counts, distances) · +MODEL / \$ SPEND also changes which model grades near the value, or triggers a paid pass. None change the rubric the judge uses.			

Audited 2026-07 · every key above is consumed in code (no dead knobs) and now round-trips through Settings (the revisionWarnAt / coldStartMin save-wipe bug is fixed).

The one indirect effect on grading — model escalation

There is exactly one way a threshold touches grading, and it's indirect: when a cheap **Haiku** screen produces a score that lands **within ± 1.5 of a decision bar**, the review is re-run on the stronger **Opus** judge (a different, more rigorous model, also chosen to counter same-model bias). A stronger model may grade a borderline case slightly differently — so moving a floor into the cluster of typical scores causes *more* borderline items to get the Opus re-grade. It still uses the **same rubric**; it does not make the rubric stricter. Today the interactive IDEA/SCRIPT/ASSETS gates escalate around the fixed copilot bar (7.5); the build-run FINAL gate escalates around your `runFloor`.

What the four review gates actually grade (the fixed rubrics)

-  **Idea gate**
- Audience fit · Demand evidence (×2) · Differentiation (×2) · Title potential · Niche coherence
 - Weighted so demand-evidence & differentiation dominate — the YouTube "inauthentic content" bar.

-  **Script gate**
- Hook (×2) · Promise-match (×2) · Beat economy · Claim hygiene · Editorial voice · Policy safety
 - + free lints: pattern-interrupt cadence (≤45s), length band (±25%)

-  **Assets gate**
- VO coverage · Visual relevance (×2) · Visual variety · Thumbnail coherence

-  **Final gate**
- Completeness · Duration band · Publish-ready (×2)
 - + Self-Watch: timing · script-match · competitive-fit · transitions

These criteria (in `rubrics.ts` / the Self-Watch gate) are the actual "grading dials." They are hard-coded and are NOT exposed as sliders — deliberately, so the judge stays calibratable against your 👍/👎 labels.

If you actually want to change HOW strictly it grades

LEVER	WHERE	EFFECT
👍 / 👎 calibration	On any QC or Self-Watch verdict (Asset Canvas checkpoints)	The intended tuning path. ~50 labels make each criterion measurable (agreement %); disputed criteria get rewritten. This is how grading improves — not the sliders.
Self-learned criteria	Shadow lessons → graduate (governed by the two <code>playbook*</code> knobs)	The system proposes new watch criteria from evidence; once confirmed they start gating. Bounded so a fluke can't become dogma.
Judge model routing	Env vars <code>QC_SCREEN_MODEL</code> / <code>QC_JUDGE_MODEL</code> / <code>VISION_MODEL</code>	A stronger judge model grades more rigorously (and costs more). This genuinely changes grading quality — see the spend map.
Rubric text	<code>rubrics.ts</code> (code change)	The literal questions the judge answers. Editing these is the direct way to change what "quality" means per gate.

Bug fixed in this pass: two knobs — `revisionWarnAt` and `coldStartMin` — were read by the pipeline but omitted from the Settings save, so **every save silently reset them to their defaults (3 and 5)**. They are now exposed in Settings, and the save **merges** onto the stored values so no knob is ever wiped. Regression tests added.